check_paired.py 사용법 설명서

> lab11 · 생물정보학실험
> 대상 파일: `lab11/src/check_paired.py`

1. 프로그램의 목적과 기능

`check_paired.py`는 **paired-end FASTQ 파일 두 개(R1, R2)가 서로 올바르게 짝을 이루고 있는지 검사하는 프로그램**입니다.

paired-end 데이터는 파일이 두 개로 나뉘어 있어서, 두 파일의 **read 개수**와 **read 순서**가 정확히 맞아야 합니다. 하나라도 어긋나면 이후 분석(trimming, 지도 정렬, 조립 등)에서 엉뚱한 read끼리 짝지어져 결과가 틀어집니다. 이 프로그램은 분석을 시작하기 전에 이것을 미리 확인해 줍니다.

프로그램이 하는 일은 다음과 같습니다.

1. R1 파일과 R2 파일을 **동시에** 한 레코드(4줄)씩 읽는다.
2. 한쪽 파일이 먼저 끝나면 → "read 개수가 다르다"고 알려 준다.
3. 각 read의 이름에서 `_1`, `_2`, `/1`, `/2` 같은 꼬리표를 떼어 낸다. → `normalize_name()`
4. 같은 순서에 있는 두 read의 이름을 비교하고, 다르면 **불일치**로 센다.
5. 검사한 read pair 수, 불일치 수, 최종 판정을 출력한다.

함수	하는 일
`normalize_name(header)`	헤더 줄에서 비교용 read 이름만 깔끔하게 뽑아낸다
`check_paired(file1, file2)`	두 파일을 함께 읽으며 개수와 이름 짝을 검사한다

2. Paired-end sequencing의 의미

DNA 조각(fragment)을 시퀀싱할 때, **한쪽 끝만 읽는 방식**을 single-end, **양쪽 끝을 모두 읽는 방식**을 paired-end라고 합니다.

```
          DNA 조각 (예: 약 300~500 bp)
5' ═══════════════════════════════════════════════ 3'
   -------->                                <--------
   R1 (앞쪽 끝에서 읽음)              R2 (반대쪽 끝에서 읽음)
```

- **R1**: 조각의 한쪽 끝에서 읽은 read → 보통 `..._1.fastq` 파일에 저장
- **R2**: 같은 조각의 반대쪽 끝에서 읽은 read → 보통 `..._2.fastq` 파일에 저장

R1 파일의 n번째 read와 R2 파일의 n번째 read는 **같은 DNA 조각에서 나온 한 쌍(pair)** 입니다. 그래서 두 파일은 항상

- read 개수가 같아야 하고,
- 같은 순서(같은 줄 위치)에 같은 조각의 read가 있어야 합니다.

paired-end의 장점은 두 read 사이의 거리를 대략 알기 때문에, 유전체에서 위치를 더 정확히 찾을 수 있고 반복 서열이 많은 곳도 더 잘 조립할 수 있다는 것입니다.

---

3. 실행 명령어와 사용 방법

`lab11` 폴더에서 R1 파일, R2 파일 순서로 경로를 넣어 실행합니다.

```
cd lab11
python3 src/check_paired.py data/SRR1553607_1.fastq data/SRR1553607_2.fastq
```

형식:

```
python3 src/check_paired.py <R1 FASTQ 파일> <R2 FASTQ 파일>
```

- 첫 번째 인자(`sys.argv[1]`) → R1 파일
- 두 번째 인자(`sys.argv[2]`) → R2 파일
- 인자를 빼먹으면 `IndexError`가 나므로 **파일 두 개를 꼭 넣어야** 합니다.
- 압축 파일(`.fastq.gz`)은 바로 읽지 못합니다. 압축을 푼 `.fastq` 파일을 넣어 주세요.

참고: FASTQ 레코드는 항상 4줄로 이루어져 있습니다.

```
@SRR1553607.1 1 length=101     ← 1줄: 헤더(read 이름)
GTTAGCGTTGTTGATCGCGACGCAACAACT…  ← 2줄: 염기 서열
+SRR1553607.1 1 length=101       ← 3줄: 구분선
BB@FFFFFHHHHHJJJJJJJJJJJJJJJJJ…  ← 4줄: 품질 점수
```

이 프로그램은 **1줄(헤더)만 비교**하고 나머지 3줄은 건너뜁니다.

---

4. R1과 R2의 read 개수를 확인하는 방법

프로그램은 두 파일을 따로따로 세지 않고, **두 파일에서 한 줄씩 동시에 읽어 나가면서** 개수를 맞춰 봅니다.

```
while True:
    h1 = f1.readline()   # R1의 헤더 줄
    h2 = f2.readline()   # R2의 헤더 줄

    if not h1 and not h2:          # 두 파일이 동시에 끝남 → 정상 종료
        break
```

```
    if not h1 or not h2:          # 한쪽만 끝남 → 개수가 다름
        print("두 파일의 read 개수가 다릅니다.")
        return
```

- `readline()`은 파일 끝에 도달하면 빈 문자열 `""`을 돌려줍니다. 그래서 `not h1`이 참이면 "R1 파일이 끝났다"는 뜻입니다.
- **둘 다 끝났다** → 개수가 같다는 뜻이므로 정상적으로 반복을 멈춥니다.
- **한쪽만 끝났다** → 한 파일에 read가 더 많다는 뜻이므로 바로 경고하고 종료합니다.

헤더를 읽은 다음에는 나머지 3줄(서열, `+`, 품질)을 건너뜁니다. 이때 줄이 모자라면 레코드가 잘린 것이므로 `"불완전한 FASTQ 레코드가 있습니다."`를 출력합니다.

```
for _ in range(3):
    if not f1.readline() or not f2.readline():
        print("불완전한 FASTQ 레코드가 있습니다.")
        return

count += 1   # 한 쌍을 다 읽었으므로 개수 +1
```

직접 개수를 세어 보고 싶다면 터미널에서 줄 수를 4로 나누면 됩니다.

```
echo $(( $(wc -l < data/SRR1553607_1.fastq) / 4 ))   # 203445
echo $(( $(wc -l < data/SRR1553607_2.fastq) / 4 ))   # 203445
```

---

5. normalize_name() 함수의 역할

```
def normalize_name(header):
    name = header.strip().split()[0]   # ① 공백 앞 첫 단어만
    name = name.lstrip("@")            # ② 맨 앞의 @ 제거

    # ③ 끝의 /1, /2 또는 _1, _2 제거
    if name.endswith(("/1", "/2", "_1", "_2")):
        name = name[:-2]

    return name
```

헤더 줄에는 read 이름 말고도 여러 정보가 붙어 있어서, 그대로 비교하면 같은 쌍인데도 "다르다"고 나올 수 있습니다. `normalize_name()`은 헤더에서 **비교에 필요한 순수한 read 이름만 남기는** 함수입니다.

단계	코드	하는 일	예시
입력			`@SRR1553607.1/1 1 length=101\n`
①	`strip().split()[0]`	줄바꿈 제거 후 공백 앞 첫 단어만	`@SRR1553607.1/1`

단계	코드	하는 일	예시
②	`lstrip("@")`	맨 앞 `@` 제거	`SRR1553607.1/1`
③	`name[:-2]`	끝 두 글자(`/1`) 제거	`SRR1553607.1`

R2 쪽 헤더 `@SRR1553607.1/2 1 length=101`도 똑같이 처리하면 `SRR1553607.1`이 되어 **두 이름이 같아집니다.**

---

6. read 이름 끝의 _1, _2 또는 /1, /2를 제거하는 이유

paired-end 데이터에서는 R1과 R2를 구분하려고 read 이름 끝에 **"몇 번째 짝인지"를 나타내는 표시**를 붙이는 경우가 많습니다.

R1 헤더	R2 헤더
`@read123/1`	`@read123/2`
`@read123_1`	`@read123_2`

이 표시는 "R1이냐 R2냐"만 알려 줄 뿐, **어떤 DNA 조각인지**는 앞부분(`read123`)이 알려 줍니다. 만약 꼬리표를 그대로 두고 비교하면

```
read123/1   ≠   read123/2      ← 같은 쌍인데 "불일치"로 잘못 판단!
```

이 되어 버립니다. 그래서 꼬리표를 떼어 내고 **공통된 이름 부분만** 비교해야 올바르게 짝을 확인할 수 있습니다.

> 참고: 이번 실습 데이터(`SRR1553607`)는 헤더가 `@SRR1553607.1 1 length=101`처럼 생겨서 꼬리표가 없습니다. 이 경우 ①단계에서 첫 단어 `SRR1553607.1`만 남기 때문에 ③단계는 아무것도 하지 않습니다. 꼬리표 제거 기능은 다른 형식의 FASTQ 파일도 검사할 수 있도록 넣어 둔 것입니다.

---

7. 같은 순서의 read가 실제로 짝을 이루는지 확인하는 과정

개수가 같다고 해서 짝이 맞는다는 보장은 없습니다. 예를 들어 한 파일만 정렬하거나, 한 파일에서만 read를 지우고 다른 read를 넣으면 개수는 같아도 순서가 어긋납니다. 그래서 프로그램은 **같은 위치의 read 이름을 하나하나 비교**합니다.

```
name1 = normalize_name(h1)
name2 = normalize_name(h2)

if name1 != name2:
    mismatch += 1
    if mismatch <= 5:                # 처음 5개까지만 화면에 보여 줌
        print("불일치:", name1, name2)
```

흐름을 그림으로 보면 다음과 같습니다.

```
  R1 파일            R2 파일            비교 결과
1번 SRR1553607.1   1번 SRR1553607.1   →  같음 ✔
2번 SRR1553607.2   2번 SRR1553607.2   →  같음 ✔
3번 SRR1553607.3   3번 SRR1553607.3   →  같음 ✔
  …                  …
```

- 불일치가 많을 수 있으므로 **처음 5개만 출력**해서 화면이 넘치지 않게 합니다. 개수(`mismatch`)는 끝까지 모두 셉니다.
- 모든 read를 다 본 뒤, 불일치가 0개면 "모든 read가 같은 순서로 짝을 이룹니다.", 1개 이상이면 "일부 read의 짝이 일치하지 않습니다."를 출력합니다.

---

8. 실제 실행 결과와 해석

8-1. 실습 데이터 (정상인 경우)

```
python3 src/check_paired.py data/SRR1553607_1.fastq data/SRR1553607_2.fastq
```

```
확인한 read pair 수: 203445
이름 불일치 수: 0
모든 read가 같은 순서로 짝을 이룹니다.
```

해석

- R1과 R2 파일 모두 read가 **203,445개**로 개수가 같습니다. ("read 개수가 다릅니다" 메시지가 나오지 않았음)
- 203,445쌍 모두 같은 위치의 read 이름이 같아서 **불일치가 0개**입니다.
- 따라서 이 두 파일은 올바른 paired-end 데이터이며, 그대로 fastp 같은 paired-end 분석 도구에 넣어도 됩니다.

8-2. 문제가 있는 경우 (연습용 작은 파일로 시험)

동작을 이해하기 위해 read 3개짜리 작은 FASTQ 파일을 만들어 일부러 문제를 넣어 보았습니다.

① **순서가 뒤바뀐 경우** — R1은 `r1, r2, r3` 순서, R2는 `r1, r3, r2` 순서

```
불일치: r2 r3
불일치: r3 r2
확인한 read pair 수: 3
이름 불일치 수: 2
일부 read의 짝이 일치하지 않습니다.
```

→ 개수는 3개로 같지만 2번째, 3번째 read의 짝이 엇갈려 있다는 것을 찾아냈습니다. `/1`, `/2` 꼬리표는 `normalize_name()`이 떼어 냈기 때문에 `r1`끼리는 정상으로 판단되었습니다.

② 개수가 다른 경우 — R1은 read 3개, R2는 read 2개

```
두 파일의 read 개수가 다릅니다.
```

→ R2 파일이 먼저 끝났기 때문에 바로 경고하고 종료합니다. 이때는 pair 수나 불일치 수가 출력되지 않습니다.

8-3. 결과 메시지 정리

출력 메시지	의미	해야 할 일
모든 read가 같은 순서로 짝을 이룹니다.	정상	다음 분석 진행
일부 read의 짝이 일치하지 않습니다.	순서가 어긋남	파일을 다시 받거나 정렬 상태 확인
두 파일의 read 개수가 다릅니다.	한쪽에 read가 더 많음	파일이 잘렸는지, 다른 파일을 넣지 않았는지 확인
불완전한 FASTQ 레코드가 있습니다.	4줄이 다 안 채워진 레코드 존재	다운로드/압축 해제가 제대로 됐는지 확인